

This project focuses on a constrained search problem: finding an exact match pattern, variably scaled, in an image array (such as a specific face or pose that appears within a crowd scene). In this assignment, we will focus on finding George P. Burdell, whose mugshot is shown in Figure 1, in an image that contains a crowd of faces, such as the sample scene in Figure 2. The faces in the scene, including George's, may be at varying distances from the camera, so they may appear scaled. For example in Figure 3a, George is scaled two times the original size and in Figure 3b, he is scaled by a factor of three. All faces will be facing forward and upright (not rotated).

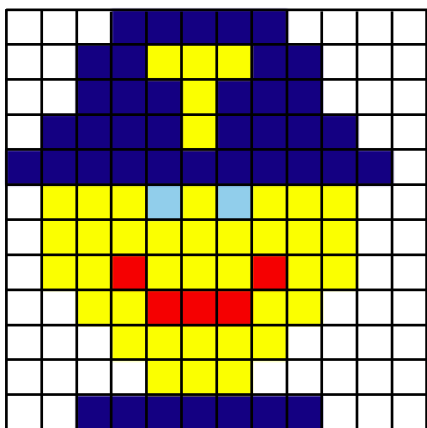


Figure 1: Mugshot of George P. Burdell

All pixels shown are within the bounding box of George. Note that the rightmost n columns of the bounding box will always contain only background pixel colors, where n is the scaling factor.

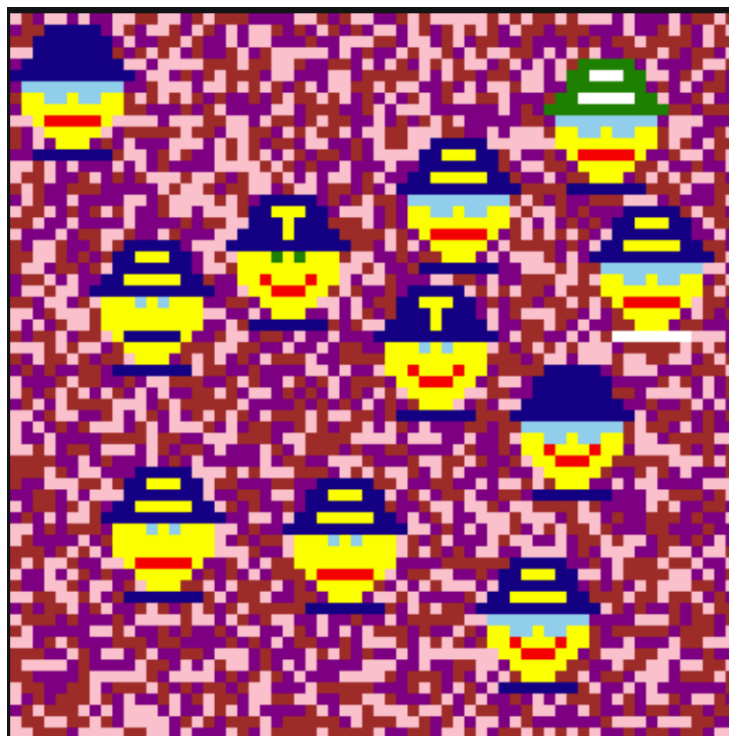


Fig. 2: Sample Crowd of Faces (test_1_1568_2283.txt)

George is found in the middle, surrounded by 5 other faces.

Disclaimer: Any resemblance to actual persons, roommates, or relatives is unintentional and coincidental.

This task needs to be performed in real-time, so the computational and storage requirements must be kept to a minimum. We are also concerned with functional correctness of the algorithm (i.e., getting the correct answer), since we do not want to generate frequent false alarms, or miss an important match.

George will appear exactly once (possibly scaled) in a random spot in each crowded scene. None of the faces in the scene will overlap with each other and all faces fit completely within the boundaries of the crowd (no partial faces hanging off the edges).

Like most college students, George has a limited wardrobe; he always wears a blue hat, with a yellow “T” for Tech, and blue shirt – and he’s always smiling. The faces that are not George will differ from him by having different color features (e.g., a green hat or white shirt) and/or structural variations (e.g., horizontal stripes instead of a “T” on hat, glasses/none, smile/flat frown).

Your task is to find the one face that exactly matches (possibly with scaling) George's mugshot shown in Figure 1.

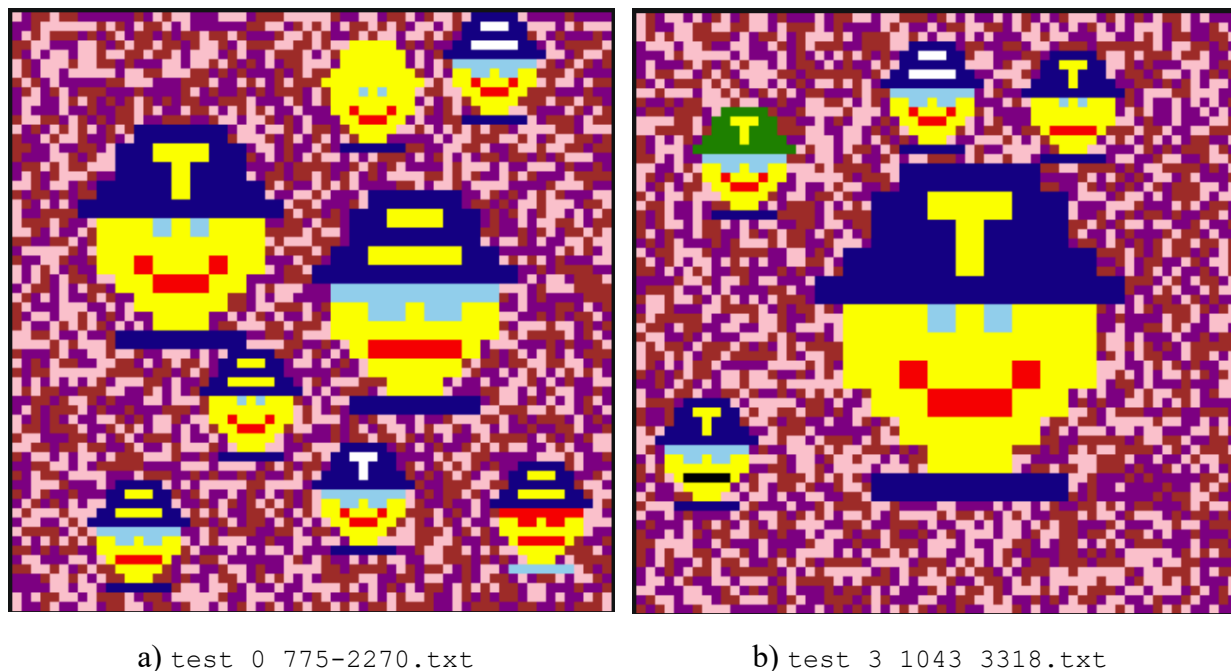


Figure 3: More Sample Crowds

The crowd scene is a 64x64 image array of pixels; each pixel is one of eleven colors whose codes are given in the color palette shown in Figure 4. Each *non-scaled* face fits in an 12x12 region of pixels. Each face may appear in the image scaled by a factor of 2, 3, 4, or 5. For example, in Figure 3a, George is scaled by a factor of 2 and fits in a 24x24 square region; in Figure 3b, he is scaled by 3 and fits in a 36x36 region. The features of the face will always have color codes in the range 1, 2, ..., 8. The background (non-face parts) of the image will always use colors whose codes are 9, 10, or 11.

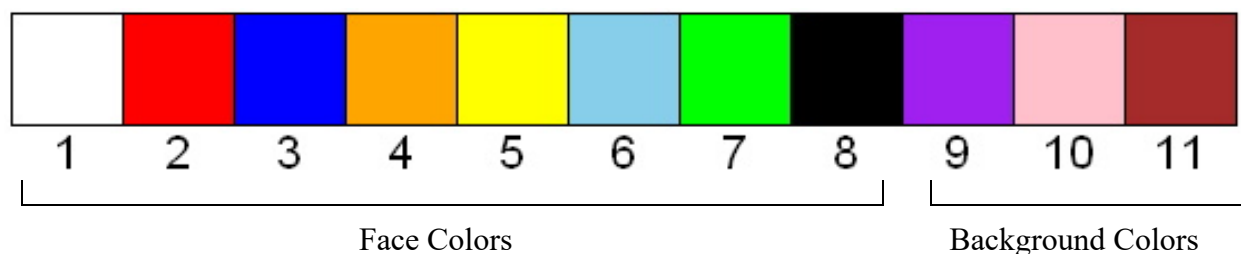


Figure 4: Color Palette

The image array is provided as input to the program as a linearized array of the pixels. The first element of the array (location 0) represents the color of the first pixel in the first row. This is followed by the second pixel (location 1) in that row, etc. The last pixel of the first row (location 63) is followed by the first pixel of the second row (location 64). This way of linearizing a two dimensional array is called *row-major order*. The color code (1-11) is packed in an unsigned byte integer for each pixel.

For example, if the top row of an image started with pixels in the three background colors (9, 10, 11, in that order) and followed by a red pixel (2), then the array would begin with a word packing 9 in byte 0, 10 in byte 1, 11 in byte 2, and 2 in byte 3. In Little Endian, this would create the word 0x020B0A09. (Recall 11 decimal is B hexadecimal and 10 decimal is A hexadecimal.)

Faces in the crowd have the following variations. Exactly one has George's combination of features matching his mugshot. The percentages indicate the probability that a face has that feature variation.

1. The face may be scaled between 1x and 5x, inclusive, space permitting.
2. The skin can be yellow (94%), orange (3%), or red (3%).
3. The shirt can be blue (85%), green (5%), white (5%), or light blue (5%).
4. The hat can have one of two emblems: "T" (60%) or horizontal stripes (40%).
5. The hat emblem color could be yellow (90%), blue (5%), or white (5%).
6. The hat color could be blue (90%), green (5%), or yellow (5%).
7. The mouth could be a smile (50%) or flat (50%).
8. The mouth color could be red (88%), blue (4%), black (4%), or orange (4%).
9. The face could have no glasses (55%) or glasses (45%). The glasses are always the same shape and are the same color as the eyes.
10. The eye color could be light blue (82%), green (6%), red (6%), or black (6%) - or green instead of light blue if the face is not George and no other feature was varied.

Objective: For this assignment, you will write two programs, one in C and one in RISC-V, to do the task described above. You should design, implement, and test your own code. There are many possible approaches you can take to solve this problem. You are encouraged to try more than one and choose the one that best trades off constraints on code size, storage, and run time.

Download the files **P1-1_code.tar.gz** and **P1-2_code.zip** from the Project 1 Canvas page. They contain the following files which will be used in the subparts of the assignment as described below.

Files for P1-1 (P1-1_code.tar.gz):	Files for P1-2 (P1-2_code.zip):
P1-1-shell.c	P1-2-shell.asm
run_tests.sh	NewRiscVAssignment.elf
tests (subfolder)	.vscode (subfolder)

Unzip each into their own folders as you did in Homework 2:

- **P1-1:** Copy **P1-1_code.tar.gz** to the remote `ece-linlabsrv01` server and uncompress and extract the file by opening a terminal view and typing this Linux command:

```
[gburdell@ece-linlabsrv01 hw2]$ tar -xvf P1-1_code.tar.gz
```

Note: *Type in the Linux commands given in this document - do not cut and paste them from the pdf since you will get errors from embedded characters that are not visible.*

- **P1-2:** Unzip **P1-2_code.zip** into a subfolder on your local machine.

Do not change any of the files within each folder. Copy `P1-1-shell.c` to a new file named `P1-1.c` in the `P1-1_code` folder, and copy `P1-2-shell.asm` to a new file named `P1-2.asm` in the `P1-2_code` folder. You will modify each of these new files to complete this assignment and submit them separately as the solutions to P1-1 and P1-2, as described below. Keep these two solution files in their respective folders so that they can be run with the proper supporting files.

Strategy: Unlike many “function only” programming tasks where a solution can be quickly envisioned and implemented, this task requires a different strategy.

1. Before writing any code, reflect on the task requirements and constraints. *Mentally* explore different approaches and algorithms, considering their potential performance and costs. The metrics of merit are **static code length**, **dynamic execution time**, and **storage requirements**. There are often trade-offs between these parameters. Sometimes back of the envelope calculations (e.g., how many comparisons or memory accesses will be performed) can help illuminate the potential of an approach.

Think of ways to avoid checking every pixel in the crowd image or matching every pixel in George’s mugshot. Try to check stronger constraints before weaker ones in order to prune the search earlier.

2. Once a promising approach is chosen, a high-level language (HLL) implementation (e.g., in C) can deepen its understanding. The HLL implementation is more flexible and convenient for exploring the solution space and should be written before constructing the assembly version where design changes are more costly and difficult to make. For P1-1, you will write a C implementation of the Find George program. Incremental development is encouraged: work on solving small parts of the problem, interleaved with testing, and gradually add to your program. Save intermediate versions of your code so you can roll back to an earlier idea if you introduce bugs or inefficiencies.
3. Once a working C version is created, it’s time to “be the compiler” and see how it translates to MIPS assembly. This is an opportunity to see how HLL constructs are supported on a machine platform (at the ISA level). This level requires the greatest programming effort; but it also uncovers many new opportunities to increase performance and efficiency. You will write the assembly version for P1-2.
4. *You'll hand in intermediate drafts P1-2-int.asm and the final version P1-2 at staggered due dates. **All assembly files should contain a change log** that has entries describing the evolution of your code as described below.*

P1-1 High Level Language Implementation:

In this part, the first two steps described above are completed. It's fine to start with a simple implementation that produces an answer; this will help deepen your understanding. Then experiment with your best ideas for a better performing solution. Each hour spent exploring here will cut many hours from the assembly level coding exercise.

You should use the simple shell C program that is provided `P1-1-shell.c` to allow you to read in a linearized image array. The shell program includes a reader function `Load_Mem()` that loads the values from a text file.

Reporting your results: *Your C program must print the index of the top left corner and the index of the bottom right corner of the square region (bounding box) containing George. For the*

example crowd shown in Figure 5 (below), it should print these locations as 1037 (the index of the top left pixel 13 pixels across and 16 rows down) and 2532 (the index of the bottom right pixel 36 pixels across and 39 rows down). This test case is provided as `test-4-1037-2532.txt`.

Note that the bounding box is square. Figure 1 shows that the rightmost column of an unscaled image will always include ONLY background colors (ANY background colors, not necessarily a solid color such as in Figure 1). If a face is scaled by n , there will be n columns of ONLY background colors on the right within its bounding box.

You can modify any part of the shell program. Just be sure that your completed assignment can read in an arbitrary crowd, find George, and correctly print his location (corners of his bounding box) since this is how you will receive points for this part of the project.

You must not change the print statement that reports the answer. Doing so will make your program fail when run by the autograder. If you would like to add more print statements as you debug your code, please wrap them in an `if` statement using a `DEBUG` flag – an example is given in the shell program – so that you can suppress printing them in the code you submit by setting `DEBUG` to 0. *If your submitted code prints extraneous output, it will be marked incorrect by the autograder.*

Grading criteria: You will not be graded for your C implementation's performance (speed and storage efficiency). Only its accuracy and good programming style will be considered (e.g., using proper data types, operations, control mechanisms, etc., and documenting your code with comments). Your C implementation does not need to use the same algorithm as the assembly program; however, it's much easier for you if it does.

Test cases: A few sample testcases have been provided, with the file naming convention: `test_n_U_L.txt`, where n is the test case number, U and L specify the upper left and lower right corners of George's bounding box as offsets into the crowd array.

As in previous assignments, we provide a short “smoke test” script (`run_tests.sh`). *Please place **run_tests.sh** in the same directory where you put your P1-1.c code and place the given test cases in a subfolder called **tests**.* You must run the smoke test on your code on the Linux Lab servers before submitting your code and fix any errors detected. If you fail to detect errors because you ran the smoke test on a different platform, it will not be a valid reason to request a regrade.

Be sure to run several more tests. The smoke tests do not represent the comprehensive set run by the autograder. You can create additional test files using ERNIE to run `P1-2-shell.asm`, just as you did in Homework 2.

1. Run `P1-2-shell.asm` with a breakpoint set on the last instruction (JALR).
2. When the emulator pauses at the breakpoint, click the Dump Memory to Clipboard button. This will save the current static memory snapshot as text that is in the format expected by the `Load_Mem` function in the `P1-1-shell.c` code.
3. Paste this saved text into a new file and name it something like “test- U - L .txt” where U and L specify the answer. The Debug Console shows the expected answer and the register `a2` also holds the answer after the final `ecall`. You can call the file whatever you like (it doesn't have to be of the form “test*.txt”).

Submitting P1-1: When you have completed the assignment, submit the single file `P1-1.c` to Canvas. You do not need to submit data files. Although it is good practice to employ a header file (e.g., `P1-1.h`) for declarations, external variables, etc., in this project you should just include this information at the beginning of your submitted program file. In order for your solution to be properly received and graded, there are a few requirements.

1. The file must be named **P1-1.c**. (Do not worry if Canvas appends a version number.)
2. Your name and the date should be included in the header comment.
3. Your submitted file should compile and execute on an arbitrary testcase provided in a test file (produced by ERNIE). It should contain the unmodified print statement, giving the solution. The command line parameters should not be altered from those used in the shell program.
4. Your program must compile and run with gcc under Linux. Compiler warnings will cause point deductions. If your program does not compile or enters an infinite loop, it will earn 0 correctness points.
5. Before submitting your code, **reset DEBUG to 0** and run your code through the smoke tests on the Linux Lab servers to ensure that your code is providing the answer in the proper format without any extraneous print statements. ***If your submitted code has a bug that the smoke test would detect, you will receive a 0.***
6. Your solution must have proper documentation (comments) and appropriate indentation.
7. Your solution must be properly uploaded to Canvas before the scheduled due date.

P1-2 Assembly Level Implementation:

In this part of the project, you will write the performance-focused assembly program that solves the Find George puzzle. A shell program (`P1-2-shell.asm`) is provided to get you started. Rename it to `P1-2.asm`. *Your solution must not change the crowd image array (do not write over the memory containing the crowd).*

Your RISC-V code must store the solution bounding box in **a1** and then call **ecall 593** which is described below.

ECALLS: A description of the RISC-V `ecall` routines you need is given below. Use the file `P1-2-shell.asm` as a starting point. There are three library routines (executed via the `ecall` instruction with an identifier stored in register `a7`, such as 593).

592: Create Scaled Crowd: This routine initializes memory beginning at the specified base address of the crowd array contained in **a0**. It sets each byte of the 4096 byte array to the corresponding pixel's color code in the 64x64 crowd image array. The color codes are defined in the palette in Fig. 3.

INPUTS: register **a0** should contain the base address (memory should already be allocated).

OUTPUTS: memory starting at the address in **a0** contains the linearized 64x64 (4096 byte) array. *Your program may NOT modify this array.*

552: Highlight Position: This routine allows you to specify an offset into the crowd array and it draws a white outline around the pixel at that offset. Pixels that have been highlighted previously in the trace are drawn with a gray outline to allow you to visually trace which positions your code has visited. For example, Figure 5 shows a white outline on pixel 161 and a gray outline on a previously highlighted pixel 160, which results from the example instructions in `P1-2-shell.asm`.

INPUTS: register **a1** should contain an offset into the Crowd array (an integer in the range 0 to 4095, inclusive). This is not the memory address, but simply the index. If you want to highlight the uppermost left pixel, this would be 0. If you want to highlight the pixel immediately to the right, the value for **a1** would be 1.

OUTPUTS: none. The visualization will show a white outline around the pixel at the index given in **a1**.

This is intended to help you debug your code. Please remove calls to highlighting `ecalls` when turning in your assignment as they will increase your static and dynamic instruction counts.

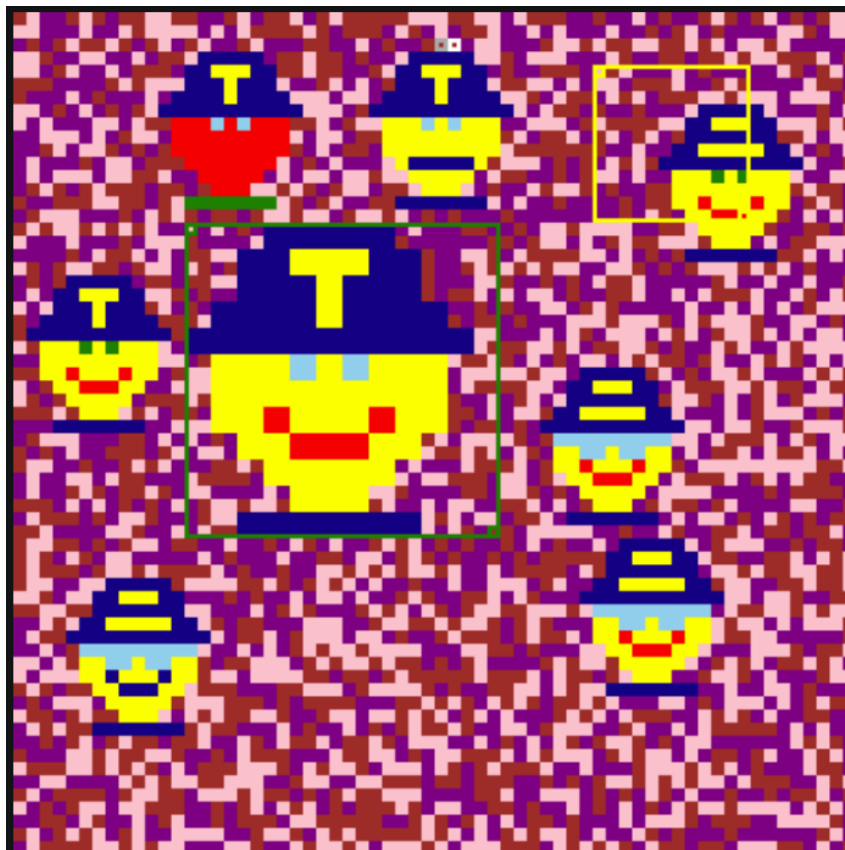


Figure 5: crowd (`test-4-1037-2532.txt`) with Correct Answer: George is located at: top left pixel 1037, bottom right pixel 2532. The green square shows the correct bounding box and the yellow shows an incorrect guess of where George is located (given to `ecall 593`). Pixels 160 and 161 are highlighted by calls to `ecall 552` in `P1-2-shell.asm`.

593: Locate Scaled George: This routine allows you to specify the position of the top left and bottom right corner pixels of George's *square* bounding box to indicate the location where George has been found.

INPUTS: register **a1** should contain two packed numbers: in the upper 16 bits, the top left corner pixel location and in the lower 16 bits, the bottom right corner pixel location. Each location should be a number between 0 and 4095, inclusive. This answer is used by an automatic grader to check the correctness of your code.

OUTPUTS: the validity of your solution will be reported in the screen view and recorded for use by the autograder. The visualization will show a green bounding box where the correct answer is and a yellow bounding box showing an incorrect answer. (To help generating testcases for P1-1, an oracle provides the correct answer in register **a2** and it is also printed in the Debug Console.)

Performance Evaluation:

In this part (P1-2), correct operation and efficient performance are both important. The assessment of your submission will include functional accuracy during 100 trials and performance and efficiency. The code size, dynamic execution length, and operand storage requirements are scored empirically, relative to a baseline solution. The baseline numbers for this project are static code size: **90** instructions, dynamic instruction length: **3000** instructions (avg.), total register and memory storage required: **25** words (not including dedicated registers `x0`, `ra`, `gp`, and not including the 1024 words for the input crowd array). *The registers used as inputs and outputs to the ecalls instructions count toward the total storage. You may use these registers for more than one purpose in optimizing your code.*

Your score will be determined through the following equation:

$$PercentCredit = 2 - \frac{Metric_{Your\ Program}}{Metric_{Baseline\ Program}}$$

Percent Credit is then used to scale the number of points for the corresponding points category. For example, if your program uses half as much storage as the baseline, then $PercentCredit = 1.5$ and the number of points for the storage category (see Project Grading table below) is 10 scaled by $1.5 = 10 * 1.5 = 15$.

Important note: while the total score for each part can exceed 100%, especially bad performance can earn *negative credit*. The sum of the combined performance metrics scores (code size, execution length, and storage) will be capped at zero; the sum of that portion of the grade will not be less than zero points. **Finally, the performance scores will be reduced by 10% for each incorrect trial (out of 100 trials). You cannot earn performance credit if your implementation fails ten or more of the 100 trials.**

In assembly language, small changes to the implementation can have a large impact on overall execution performance. Often trade-offs arise between static code size, dynamic execution length, and operand storage requirements. Creative approaches and a thorough understanding of the algorithm and programming mechanisms will often yield impressive results. Almost all engineering problems require multidimensional evaluation of alternative design approaches. Knowledge and creativity are critical to effective problem solving.

Submitting your solution: For your P1-2 solution to be properly received and graded, there are a few requirements.

1. **Checkpoint1:** A first draft version of your code should be submitted in a file named `P1-2-first-draft.asm`. This is due before the final version and will not be graded for accuracy or efficiency. It must contain a *change log*: a brief description of changes made from `P1-2-shell.asm` to this version of code. If this code does not incorporate substantive changes made to the shell code, you will not receive credit for this checkpoint. Here are some example entries:


```
# CHANGE LOG: brief description of changes made from P1-2-shell.asm
# to this version of code.
# Date Modification
# 09/24 Looping through pixels to find one w/ color ...      (example)
# 09/28 Reduced avg DI by only looking at pixels starting at row ...
```
2. **Checkpoint2:** About one week before the final due date, a short graded survey will ask you to describe your progress. Be sure to continue adding to the change log as you improve your solution.
3. The final version file must be named **P1-2.asm**. *It must also contain a change log that records a brief description of changes made from the previously submitted intermediate P1-2-first-draft version.* (As mentioned above, it is OK if Canvas renames it slightly with an extended version number; we'll grade your most recent submission.)
4. Your name and the date should be inserted at the beginning of the file.
5. Your final program must produce and store the correct value in **a1** and use **ecall 593** to report the answer.
6. Your final program must return to the operating system via the **j alr** instruction.
7. *Programs that include infinite loops or produce simulator warnings or errors will receive zero credit.*
8. Your solution must include proper documentation.
9. Your intermediate and final solutions must be properly uploaded to Canvas before the scheduled due dates.

Project Grading: The project grade will be determined as follows:

<i>part</i>	<i>description</i>	<i>percent</i>
P1-1	Find George (C code)	25
P1-2	Find George (RISC-V assembly)	
	checkpoints, correct operation, proper commenting, change log	25
	static code size	15
	dynamic execution length	25
	operand storage requirements	10
	<i>total</i>	<i>100</i>

All code (MIPS and C) must be documented for full credit.

Honor Policy: In all programming assignments, you must design, write, and debug your solutions individually. You must not accept/copy/solicit code from, share code with, debug code, or discuss its performance with any AI assistant or any person, except the instructors/TAs. Once you begin implementing your solution, you must work alone except for support from the teaching staff. Any submitted assignment containing non-shell code that

is not fully created and debugged by the student constitutes academic misconduct. You must not share any code, homework solution or any graded work *before or after* the due date.

Students are responsible for understanding all of the detailed explanation of this policy linked in the syllabus in the last sentence under ACADEMIC HONESTY: “Academic Integrity Specifics for ECE2035.”

Good luck and happy coding!